

Relazione Paradigmi di programmazione

Simone Nardella

1 Classi astratte

All'interno del package si trovano gli oggetti "Articolo" e "Lista" sono accompagnati da una versione astratta. Queste 2 classi hanno il dovere di fornire uno "stampo" alle implementazioni possibili (nel nostro caso "ArticoloSpesa" e "ListaSpesa").

1.1 FormatoArticolo

Fornisce tutte le funzioni minime che un articolo deve possedere. Alcuni metodi posseggono il "throws exceptions.GenericException" per permettere ai figli della classe di lanciare eccezioni diverse dal padre. La classe implementa le interfacce "Cloneable" e "Serializable" per imporre alle classi figlie di essere serializzabili (in modo da poter essere salvate su file) e clonabili (per permettere alla classe lista di fornire l'oggetto articolo senza che l'originale possa essere modificato esternamente).

Metodi:

checkString: Controlla se una data stringa sia adatta ad essere un nome per un articolo.

Parametri:

- **str:** La stringa su cui applicare i controlli.

Restituisce:

- **false:** Se la stringa fornita è "null" o vuota.

- **true:** In tutti gli altri casi.

```
public static boolean checkString(String nome){
    if(str == null || str.isBlank()){
        return false;
    }
    return true;
}
```

FormatoArticolo: Costruttore della classe. Ad ogni parametro è controllata la sua validità. Se la categoria fornita non è valida viene impostata al valore di default. Il costruttore esegue i controlli minimi, risparmiandoli alle future classi figlie.

Parametri:

- **nome:** Il nome che si vuole dare all'articolo.
- **costo:** Il costo dell'articolo per singolo pezzo.
- **categoria:** La categoria dell'articolo.
- **quantità:** La quantità dell'articolo.

Eccezioni:

- **ParametroException:** Se uno dei seguenti parametri non è valido.

```
public FormatoArticolo(String nome, float costo,
    String categoria, int quantita) throws
    ParametroException {
    if(!checkString(nome)){
        throw new exceptions.ParametroException(
            "nome non valido");
    }

    this.nome = nome.trim();
    this.costo = costo;
```

```

        if(!checkString(categoria)) {
            this.categoria = CATEGORIA_DEFAULT;
        }
        else {
            this.categoria = categoria.trim();
        }
        this.quantita = quantita;
    }

```

E' possibile evocare il costruttore senza specificare Categoria e/o Quantità impostandole così al valore di default:

Costruttore 2 (senza quantità e categoria):

```

public FormatoArticolo(String nome, float costo)
    throws ParametroException {
    this(nome, costo, CATEGORIA_DEFAULT,
        QUANTITA_DEFAULT);
}

```

Costruttore 3 (senza quantità):

```

public FormatoArticolo(String nome, float costo,
    String categoria) throws ParametroException
{
    this(nome, costo, categoria,
        QUANTITA_DEFAULT);
}

```

Costruttore 4 (senza categoria):

```

public FormatoArticolo(String nome, float costo,
    int quantita) throws ParametroException {
    this(nome, costo, CATEGORIA_DEFAULT,
        quantita);
}

```

Getters:

getNome: Restituisce il nome dell'articolo.

```
public String getNome() {  
    return nome;  
}
```

getCosto: Restituisce il costo dell'articolo.

```
public float getCosto() {  
    return costo;  
}
```

getQuantità: Restituisce la quantità dell'articolo.

```
public int getQuantita() {  
    return quantita;  
}
```

getCategoria: Restituisce la categoria dell'articolo.

```
public String getCategoria() {  
    return categoria;  
}
```

Setters:

tutti i setters restituiscono l'oggetto stesso per permettere la concatenazione di più funzioni da parte del programmatore.

setNome: Imposta un nuovo nome all'articolo.

Parametri:

- **newNome:** Il nuovo nome che si vuole assegnare

Restituisce:

- **l'oggetto stesso.**

Eccezioni:

- **ParametroException:** Se la stringa fornita è "null" oppure vuota.
- **GenericException:** Per permettere alle classi figlie di lanciare eccezioni piu' specifiche.

```
public FormatoArticolo setNome(String newNome)
    throws ParametroException, exceptions.
    GenericException {
        if(!checkString(newNome)) {
            throw new ParametroException("nome non
            valido");
        }
        this.nome = newNome.trim();
        return this;
    }
```

setCategoria: Imposta una nuova categoria all'articolo.

Parametri:

- **newCategoria:** La nuova categoria che si vuole assegnare.

Restituisce:

- **l'oggetto stesso.**

Eccezioni:

- **ParametroException:** Se la stringa fornita è null oppure vuota.
- **GenericException:** Per permettere alle classi figlie di lanciare eccezioni piu' specifiche.

```

public FormatoArticolo setCategoria(String
    newCategoria) throws ParametroException,
    exceptions.GenericException{
    if (!FormatoArticolo.checkString(
        newCategoria)) {
        throw new ParametroException("categoria_
            non_è valida");
    }
    this.categoria = newCategoria.trim();
    return this;
}

```

setCosto: Imposta un nuovo costo all'articolo.

Parametri:

- **newCosto:** Il nuovo costo da assegnare.

Restituisce:

- **l'oggetto stesso.**

Eccezioni:

- **GenericException:** Per permettere alle classi figlie di lanciare eccezioni più specifiche.

```

public FormatoArticolo setCosto(float newCosto)
    throws exceptions.GenericException {
    this.costo = newCosto;
    return this;
}

```

setQuantità: Imposta una nuova quantità all'articolo.

Parametri:

- **newQuantità:** La nuova quantità che si vuole assegnare.

Restituisce:

- l'oggetto stesso.

Eccezioni:

- **GenericException:** Per permettere alle classi figlie di lanciare eccezioni piu' specifiche.

```
public FormatoArticolo setQuantita(int
    newQuantita) throws exceptions.
    GenericException{
        this.quantita = newQuantita;
        return this;
    }
```

toString: Restituisce una rappresentazione a Stringa dell'oggetto.

Restituisce:

- la stringa rappresentante l'oggetto.

```
@Override
public String toString() {
    return ("Nome:␣" + this.nome + "\tCosto:␣" +
        this.costo + "\tCategoria:␣" + this.
        categoria + "\tQuantita':␣" + this.
        quantita);
}
```

equals: Indica se questo oggetto è equivalente a quello fornito.

Parametri:

- **articolo:** l'oggetto che si vuole comparare.

Restituisce:

- **true:** Se l'oggetto coincide.
- **false:** Altrimenti.

```

@Override
public boolean equals(Object articolo) {
    if(!(articolo instanceof FormatoArticolo)) {
        return false;
    }
    FormatoArticolo temp = (FormatoArticolo)
        articolo;
    if(temp.getNome().equalsIgnoreCase(this.nome
    )) {
        return true;
    }
    return false;
}

```

myClone: Restituisce un clone dell'oggetto. Essendo "FormatoArticolo" una classe generica, il metodo è astratto.

Restituisce:

- **una copia dell'oggetto.**

Eccezioni:

- **ParametroException:** Se vi è un errore nella costruzione del clone.

```

abstract public FormatoArticolo myClone() throws
    ParametroException;

```

matchCategoria: Controlla la validità dell'input e che la stringa fornita coincida con la categoria dell'articolo ignorando il case.

Parametri:

- **categoria:** la stringa da comparare alla categoria dell'articolo.

Restituisce:

- **true:** se coincidono.

- **false:** altrimenti.

Eccezioni:

- **ParametroException:** Se la stringa fornita è "null" oppure vuota.

```
public Boolean matchCategoria(String categoria)
    throws ParametroException {
    if(!checkString(categoria)) {
        throw new ParametroException("categoria_
        non_è_valida");
    }
    if(categoria.trim().equalsIgnoreCase(this.
        categoria)) {
        return true;
    }
    return false;
}
```

1.2 FormatoLista

Fornisce tutte le funzioni minime che una lista deve possedere. Alcuni metodi posseggono il "throws exceptions.GenericException" per permettere ai figli di lanciare eccezioni diverse dal padre. La classe implementa l'interfaccia "Serializable" per imporre alle classi figlie di essere serializzabili (in modo da poter essere salvate su file).

Metodi:

FormatoLista: Costruttore della classe. Al nome della lista viene controllata la validità. Il costruttore esegue i controlli minimi, risparmiandoli alle future classi figlie.

Parametri:

- **nome:** Il nome che si vuole dare alla lista.

Eccezioni:

- **ParametroException:** Se il parametro non è valido.

```
public FormatoLista(String nome) throws
    ParametroException {
    if(nome == null || nome.isBlank()) {
        throw new exceptions.ParametroException(
            "nome non valido");
    }
    else {
        this.nome = nome.trim();
    }
}
```

getNome: Restituisce il nome della lista.

```
public String getNome() {
    return this.nome;
}
```

setNome: Imposta un nuovo nome alla lista.

Parametri:

- **newNome:** Il nuovo nome da impostare.

Eccezioni:

- **ParametroException:** Se newNome è "null" oppure vuota.

```
public void setName(String newNome) throws
    ParametroException {
    if(newNome == null || newNome.isBlank()) {
        throw new exceptions.ParametroException(
            "nome non valido");
    }
}
```

```

    }
    this.nome = newNome;
}

```

Tutti gli altri metodi della classe risultano astratti, poichè dovrebbero operare sulla struttura dati che salva gli articoli (la lista effettiva). Ho preferito perciò presentarli, insieme alla loro implementazione, nella sezione dedicata alla classe "listaSpesa".

2 Package "Programma":

Questo package contiene il codice vero e proprio del progetto. Al suo interno troviamo le implementazioni della classi astratte, indirizzate verso una comune lista della spesa del supermercato.

2.1 ArticoloSpesa

Versione orientata alla spesa di "FormatoArticolo". Rispetto al padre ha regole più stringenti sui possibili valori dei campi per l'articolo:

- La quantità di default è 1.
- La quantità deve essere necessariamente maggiore o uguale ad 1.
- L'articolo non può avere un prezzo minore di 0.
- L'articolo non supporta l'operazione "setNome".

Ho deciso di non implementare la funzione "setNome" poichè non specificato dal testo progettuale. Questa scelta non riguarda la classe astratta "FormatoArticolo" che, essendo generica, ho ritenuto dovesse avere tutte le funzioni disponibili. Se un domani fosse necessario implementare questa funzione anche per "ArticoloSpesa" basterà semplicemente cancellare il corpo del metodo, dato che "ArticoloSpesa" è figlia di "FormatoArticolo".

Metodi:

Dato che molti metodi sfruttano l'implementazione dal padre senza aggiungere altro, elencherò solo i metodi nuovi o implementati diversamente.

checkCosto: Controlla che il valore fornito sia adatto per essere un costo di ArticoloSpesa.

Parametri:

- **costo** il valore da controllare.

Restituisce:

- **false:** Se il valore fornito è ≤ 0 .
- **true:** In tutti gli altri casi.

```
public static boolean checkCosto(float costo) {
    if(costo <=0) {
        return false;
    }
    return true;
}
```

checkQuantita: Controlla che la quantità fornita sia adatta per un ArticoloSpesa.

Parametri:

- **quantita** il valore da controllare.

Restituisce:

- **false:** Se il valore fornito è < 1 .
- **true:** In tutti gli altri casi.

```
public static boolean checkQuantita(int quantita
) {
    if(quantita < 1) {
        return false;
    }
}
```

```

        return true;
    }

```

ArticoloSpesa: Costruttore della classe. Ad ogni parametro viene controllata la validità, se la categoria fornita non è valida, viene impostata al valore di default. Rispetto al padre, ha un ulteriore controllo di validità sui campi "costo" e "quantità".

Parametri:

- **nome:** Il nome che si vuole dare all'articolo.
- **costo:** Il costo dell'articolo per singolo pezzo.
- **categoria:** La categoria dell'articolo.
- **quantità:** La quantità dell'articolo.

Eccezioni:

- **ParametroException:** Se uno dei seguenti parametri non è valido.

```

public ArticoloSpesa(String nome, float costo,
    String categoria, int quantita) throws
    ParametroException {
    super(nome, costo, categoria, quantita);
    if(!checkCosto(costo)) {
        throw new exceptions.ParametroException(
            "costo non valido");
    }
    this.costo = (float) (Math.floor(costo *
        100) / 100);
    if(!checkQuantita(quantita)) {
        throw new exceptions.ParametroException(
            "quantita' non valida");
    }
    else {
        this.quantita = quantita;
    }
}

```

come il padre, ArticoloSpesa possiede i costruttori privi di "Categoria" e/o "quantità" che ometto per questioni di brevità.

setName: ArticoloSpesa non supporta questa operazione.

Parametri:

- **newNome** il nuovo nome da impostare.

Eccezioni:

- **OperazioneNonSupportataException:** viene lanciata all'avvio della funzione.

```
@Override
public ArticoloSpesa setName(String newNome)
    throws OperazioneNonSupportataException {
    throw new exceptions.
        OperazioneNonSupportataException("non_è'_
        permesso_cambiare_nome");
}
```

setQuantita: Rispetto al padre, il valore fornito deve essere necessariamente > 1.

Parametri:

- **newQuantita** La nuova quantità che si vuole assegnare.

Restituisce:

- **L'oggetto stesso.**

Eccezioni:

- **ParametroException:** se newQuantita < 1.

```
@Override
public ArticoloSpesa setQuantita(int newQuantita
    ) throws ParametroException {
```

```

        if(!checkQuantita(newQuantita)) {
            throw new exceptions.ParametroException(
                "nuova_quantita' non valida");
        }
        quantita = newQuantita;
        return this;
    }

```

setCosto: Rispetto al padre, tutti i valori vengono arrotondati per difetto alla seconda cifra decimale. Il valore fornito deve essere necessariamente > 0 .

Parametri:

- **newCosto** Il nuovo costo da assegnare.

Restituisce:

- **L'oggetto stesso.**

Eccezioni:

- **ParametroException:** se $\text{newQuantita} \leq 0$.

```

@Override
public ArticoloSpesa setCosto(float newCosto)
    throws ParametroException {
    if(!checkCosto(newCosto)) {
        throw new exceptions.ParametroException(
            "costo non valido");
    }
    //tronca il costo alla seconda cifra
    decimale
    this.costo = (float) (Math.floor(newCosto *
        100) / 100);
    return this;
}

```

myClone: Restituisce una copia dell'oggetto stesso.

Restituisce:

- **clone:** Una copia dell'oggetto.

Eccezioni:

- **ParametroException:** Se vi è un errore nella costruzione del clone.

```
@Override
public ArticoloSpesa myClone() throws
    ParametroException{
    ArticoloSpesa clone = new ArticoloSpesa(this
        .nome, this.costo, this.categoria, this.
        quantita);
    return clone;
}
```

2.2 ListaSpesa

Versione orientata alla spesa di "FormatoArticolo". Implementa tutti i metodi del padre.

ListaSpesa: Costruttore della classe. Inizializza la struttura dati per salvare gli "ArticoliSpesa".

Parametri:

- **nome:** Il nome da dare alla lista.

Eccezioni:

- **ParametroException:** Se nome è "null" oppure vuoto.

```
public ListaSpesa(String nome) throws
    ParametroException {
```

```

        super(nome);
        lista = new ArrayList <ArticoloSpesa>();
    }

```

aggiungi: Aggiunge un articolo alla lista.

Parametri:

- **articolo:** L'articolo da aggiungere (che deve necessariamente estendere "FormatoArticolo").

Eccezioni:

- **ParametroException:** Se articolo non è di tipo "ArticoloSpesa".

```

public <Param extends FormatoArticolo> void
    aggiungi(Param articolo) throws exceptions.
    ParametroException, RidondanzaException {
    if(articolo.getClass() != ArticoloSpesa.
        class) {
        throw new exceptions.ParametroException
            ();
    }
    if(lista.contains(articolo)) {
        throw new exceptions.RidondanzaException
            ("articolo_gia'_presente_in_lista");
    }
    lista.add((ArticoloSpesa) articolo);
}

```

aggiungiAll: Aggiunge una collezione di articoli alla lista.

Parametri:

- **articoliDaAggiungere:** La collezione di articoli da aggiungere (gli oggetti all'interno della collezione devono estendere la classe "FormatoArticolo").

Eccezioni:

- **ParametroException:** Se uno o più elementi della collezione non sono di tipo ArticoloSpesa o hanno valori "null".
- **RidondanzaException:** Se uno o più articoli della collezione è già presente in lista oppure se la collezione ha elementi ridondanti.

```

@Override
public <Param extends FormatoArticolo> void
    aggiungiAll(Collection<Param>
        articoliDaAggiungere) throws
        ParametroException, RidondanzaException {
    ArrayList<ArticoloSpesa>
        articoliDaAggiungereCastati = new
        ArrayList<>();
    for(Object element : articoliDaAggiungere) {
        if(element.getClass() != ArticoloSpesa.
            class || element == null) {
            throw new exceptions.
                ParametroException("Tipologia_
                    oggetto_terrata");
        }
        articoliDaAggiungereCastati.add((
            ArticoloSpesa) element);
    }

    for(ArticoloSpesa element :
        articoliDaAggiungereCastati) {
        if(lista.contains(element)) {
            lista.removeAll(
                articoliDaAggiungereCastati);
            throw new RidondanzaException("
                elemento_gia'_presente_in_lista")
                ;
        }
        else {
            lista.add(element);
        }
    }
}

```

```
}
```

size: Restituisce la dimensione della lista.

Restituisce:

- **La dimensione della lista.**

```
@Override
public int size() {
    return lista.size();
}
```

contains: Indica se l'articolo fornito è presente in lista.

Parametri:

- **articolo:** L'articolo da cercare.

Restituisce:

- **true:** Se presente.
- **false:** Altrimenti.

```
@Override
public <Param extends FormatoArticolo> boolean
    contains(Param articolo) {
    return lista.contains(articolo);
}
```

contains (per nome): Stesso metodo ma richiede il nome dell'articolo invece che l'oggetto.

```
public <Param extends FormatoArticolo> boolean
    contains(String nomeArticolo) {
    for(ArticoloSpesa element : lista) {
```

```

        if (element.getNome().equals(
            nomeArticolo)){
            return true;
        }
    }
    return false;
}

```

getArticolo (per indice): Restituisce una copia dell'articolo all'indice specificato.

Parametri:

- **index:** La posizione dell'oggetto da restituire.

Eccezioni:

- **CloneNotSupportedException:** Se l'oggetto in lista non supporta *Clone*.
- **IndexOutOfBoundsException:** Se l'index fornito è < 0 oppure $> \text{lista.size}$.

```

@Override
public ArticoloSpesa getArticolo(int index)
    throws CloneNotSupportedException,
    ParametroException {
    return lista.get(index).myClone();
}

```

getArticolo (per nome): Restituisce una copia dell'articolo con il nome specificato.

Parametri:

- **nomeArticolo:** il nome dell'articolo che si vuole ottenere.

Eccezioni:

- **CloneNotSupportedException:** Se l'oggetto in lista non supporta *Clone*.
- **ParametroException:** se vi sono problemi nella creazione della copia.

Restituisce:

- **L'articolo richiesto.**

```
public ArticoloSpesa getArticolo(String
    nomeArticolo) throws
    CloneNotSupportedException,
    ParametroException {
    for(ArticoloSpesa element : lista) {
        if(element.getNome().equals(nomeArticolo)
        )) {
            return element.myClone();
        }
    }
    return null;
}
```

ricercaCategoria: Restituisce tutti gli articoli con una certa categoria.

Parametri:

- **str:** la categoria con cui cercare.

Eccezioni:

- **ParametroException:** se la stringa fornita è "null" oppure vuota.

Restituisce:

- **La lista di articoli aventi quella categoria.**

```
@Override
public ArrayList<ArticoloSpesa> ricercaCategoria
    (String str) throws ParametroException {
```

```

        ArticoloSpesa.checkString(str);
        ArrayList<ArticoloSpesa> trovati = new
            ArrayList<ArticoloSpesa>();
        for(ArticoloSpesa elemento : lista) {
            if (elemento.matchCategoria(str)) {
                trovati.add(elemento.myClone());
            }
        }
        return trovati;
    }
}

```

ricercaNome: Restituisce tutti gli articoli il cui nome inizia con una certa stringa. Se il nome fornito è vuoto vengono restituiti tutti gli articoli della lista.

Parametri:

- **str:** Le iniziali con cui cercare.

Eccezioni:

- **ParametroException:** Se la stringa fornita è "null".

Restituisce:

- **Tutti gli articoli che iniziano per "str".**

```

@Override
public ArrayList<ArticoloSpesa> ricercaNome(
    String str) throws ParametroException {
    if(str == null) {
        throw new ParametroException();
    }
    ArrayList<ArticoloSpesa> trovati = new
        ArrayList<ArticoloSpesa>();
    for(ArticoloSpesa elemento : lista) {
        if (elemento.getNome().toLowerCase().
            startsWith(str.toLowerCase())) {
            trovati.add(elemento);
        }
    }
}

```

```

        }
    }
    return trovati;
}

```

eliminaPerNome: Elimina un articolo dalla lista in base al suo nome.

Parametri:

- **nomeArticolo:** Il nome dell'articolo da eliminare.

Eccezioni:

- **ParametroException:** Se la stringa fornita è "null" oppure vuota.

Restituisce:

- **true:** Se la lista conteneva l'elemento indicato.
- **false:** Altrimenti.

```

@Override
public boolean eliminaPerNome(String
    nomeArticolo) throws ParametroException {
    if(nomeArticolo.isBlank()) {
        throw new ParametroException();
    }
    ArticoloSpesa daRimuovere = null;
    for(ArticoloSpesa elemento : lista) {
        if (elemento.getNome().equalsIgnoreCase(
            nomeArticolo)){
            daRimuovere = elemento;
        }
    }
    return lista.remove(daRimuovere);
}

```

eliminaPerCategoria: Elimina tutti gli articoli appartenenti a una certa categoria.

Parametri:

- **nomeCategoria:** La categoria per cui eliminare.

Eccezioni:

- **ParametroException:** Se la stringa fornita è "null" oppure vuota.

Restituisce:

- **Il numero di elementi eliminati.**

```
public int eliminaPerCategoria(String
    nomeCategoria) throws ParametroException {
    ArrayList<ArticoloSpesa> daRimuovere = new
        ArrayList<ArticoloSpesa>();
    for(ArticoloSpesa elemento : lista) {
        if (elemento.matchCategoria(
            nomeCategoria)){
            daRimuovere.add(elemento);
        }
    }
    lista.removeAll(daRimuovere);
    return daRimuovere.size();
}
```

eliminaCategoria: imposta alla categoria di default ("non categorizzato") tutti gli articoli appartenenti alla categoria fornita.

Parametri:

- **nomeCategoria:** La categoria da eliminare.

Eccezioni:

- **ParametroException:** Se la stringa fornita è "null" oppure vuota.

Restituisce:

- Il numero di articoli a cui viene eliminata la categoria.

```
public int eliminaCategoria(String nomeCategoria
    ) throws ParametroException {
    int n = 0;
    for(ArticoloSpesa elemento : lista) {
        if (elemento.matchCategoria(
            nomeCategoria)){
            elemento.setCategoria(ArticoloSpesa.
                CATEGORIA_DEFAULT);
            n++;
        }
    }
    return n;
}
```

costoTotale: Calcola il costo totale degli articoli nella lista (tenendo conto anche delle quantità).

Restituisce:

- Il costo totale della lista.

```
@Override
public float costoTotale() {
    float totale = 0;
    for(ArticoloSpesa element : lista) {
        totale += element.getCosto();
    }
    return totale;
}
```

MaggioriDi: Trova tutti gli articoli con un costo maggiore di un certo valore.

Parametri:

- **x:** Il costo di cui gli articoli devono essere maggiori.

Eccezioni:

- **ParametroException:** Se "x" <= 0.

Restituisce:

- **Gli articoli con costo maggiore di "x".**

```
@Override
public ArrayList<ArticoloSpesa> MaggioriDi(float
    prezzo) {
    ArrayList<ArticoloSpesa> trovati = new
        ArrayList<>();
    for(ArticoloSpesa elemento : lista) {
        if (elemento.getCosto() > prezzo){
            trovati.add(elemento.myClone());
        }
    }
    return trovati;
}
```

MinoriDi: Trova tutti gli articoli con un costo inferiore a un certo valore.

Parametri:

- **x:** Il costo di cui gli articoli devono essere minori.

Eccezioni:

- **ParametroException:** Se "x" <= 0.

Restituisce:

- **Gli articoli con costo minore di "x".**

```
@Override
public ArrayList<ArticoloSpesa> MinoriDi(float
    prezzo) {
```

```

        ArrayList<ArticoloSpesa> trovati = new
            ArrayList<>();
        for(ArticoloSpesa elemento : lista) {
            if (elemento.getCosto() < prezzo){
                trovati.add(elemento.myClone());
            }
        }
        return trovati;
    }
}

```

CompresiTra: Trova tutti gli articoli con un costo compreso tra due valori.

Parametri:

- **x:** Il costo di cui gli articoli devono essere maggiori o uguali.
- **y:** Il costo di cui gli articoli devono essere minori o uguali.

Eccezioni:

- **ParametroException:** Se "x" o "y" sono ≤ 0 oppure se "y" $< "x"$.

Restituisce:

- **Gli articoli con costo compreso tra "x" e "y".**

```

@Override
public ArrayList<ArticoloSpesa> CompresiTra(
    float prezzoMin, float prezzoMax) {
    ArrayList<ArticoloSpesa> trovati = new
        ArrayList<>();
    for(ArticoloSpesa elemento : lista) {
        if (elemento.getCosto() <= prezzoMax &&
            elemento.getCosto() >= prezzoMin) {
            trovati.add(elemento.myClone());
        }
    }
    return trovati;
}
}

```

iterator: Restituisce un iteratore per gli articoli della lista.

Restituisce:

- Un iteratore della lista.

```
@Override
public Iterator<ArticoloSpesa> iterator() {
    return lista.iterator();
}
```

toString: Restituisce una stringa rappresentativa della lista e dei suoi contenuti.

Restituisce:

- Una stringa rappresentante l'oggetto.

```
@Override
public String toString() {
    return lista.toString();
}
```

2.3 GestioneListaSpesa

Classe statica, fornisce tutte le funzioni per la gestione delle varie liste. Dato che è un gestore e fornisce solo "servizi" non ha senso che possa essere istanziata, ma, poichè solo le nested-class possono essere rese "static", vi è una seconda classe interna "GestioneListe". GestioneListe ha:

- *Modificatore final*: per impedirne l'estensione, dato che estendere una classe statica non ha senso.
- *Costruttore privato*: per evitare l'istanziamento, dato che istanziare una classe statica non ha senso.
- *Tutte le funzioni statiche*: non potendo essere istanziata, tutti i metodi non statici sarebbero inaccessibili.

reset: Cancella tutte le informazioni immagazzinate nel gestore (liste e categorie).

```
public static void reset() {  
    listone.clear();  
    elencoCategorie.clear();  
}
```

checkString: Controlla la validità di una stringa.

Parametri:

- **str:** La stringa da valutare.

Eccezioni:

- **ParametroException:** Se la stringa fornita è "null" oppure vuota.

```
@Override  
public ArrayList<ArticoloSpesa> MaggioriDi(float  
    prezzo) {  
    ArrayList<ArticoloSpesa> trovati = new  
        ArrayList<>();  
    for(ArticoloSpesa elemento : lista) {  
        if (elemento.getCosto() > prezzo){  
            trovati.add(elemento.myClone());  
        }  
    }  
    return trovati;  
}
```

creaListaSpesa: Crea una nuova lista della spesa.

Parametri:

- **nome:** Il nome della lista che si vuole creare.

Eccezioni:

- **ParametroException:** Se il nome della lista è "null" oppure vuoto.
- **RidondanzaException:** Se è già presente una lista con lo stesso nome.

```
public static void creaListaSpesa(String nome)
    throws ParametroException,
    RidondanzaException {
    checkString(nome);
    if(containsLista(nome.trim())) {
        throw new RidondanzaException("lista_
        'presente");
    }
    ListaSpesa lista = new ListaSpesa(nome.trim
    ());
    listone.add(lista);
}
```

cambiaNomeListaSpesa: Cambia il nome della lista della spesa.

Parametri:

- **nomeLista:** Il nome della lista da cambiare.
- **newNome:** Il nuovo nome da assegnare.

Eccezioni:

- **ElementoNonTrovatoException:** Se la lista da modificare non esiste.
- **ParametroException:** Se una delle due stringhe non è adatta come nome lista.

```
public static void cambiaNomeListaSpesa(String
    nomeLista, String newNome) throws
    ElementoNonTrovatoException,
    ParametroException {
```

```

        getListaSpesa(nomeLista).setNome(newNome.
            trim());
    }

```

eliminaListaSpesa: Elimina la lista spesa avente il nome specificato.

Parametri:

- **nome:** Il nome della lista che si vuole eliminare.

Eccezioni:

- **ParametroException:** Se la stringa fornita non è adatta come nome lista.

Restituisce:

- **true:** Se la lista conteneva l'elemento indicato.
- **false:** Altrimenti.

```

public static boolean eliminaListaSpesa(String
    nome) throws ParametroException{
    if(!containsLista(nome)) {
        return false;
    }
    try {
        return listone.remove(getListaSpesa(
            nome));
    }
    catch (ElementoNonTrovatoException e) {
        return false;}
}

```

containsLista: Verifica se esiste una lista con il nome fornito.

Parametri:

- **nome:** Il nome della lista da cercare.

Eccezioni:

- **ParametroException:** Se la stringa fornita è "null" oppure vuota.

Restituisce:

- **true:** Se la lista conteneva l'elemento indicato.
- **false:** Altrimenti.

```
public static boolean containsLista(String nome)
    throws ParametroException {
    for(ListaSpesa element : listone) {
        if(element.getNome().equalsIgnoreCase(
            nome.trim())) {
            return true;
        }
    }
    return false;
}
```

getListaSpesa: Restituisce la lista della spesa con il nome indicato.

Parametri:

- **nomeLista:** Il nome della lista che si vuole ottenere.

Eccezioni:

- **ParametroException:** Se la stringa fornita è "null" oppure vuota.
- **ElementoNonTrovatoException:** Se la lista indicata non esiste.

Restituisce:

- **La lista con il nome specificato.**

```

public static ListaSpesa getListaSpesa(String
    nomeLista) throws ParametroException,
    ElementoNonTrovatoException {
    checkString(nomeLista);
    for(ListaSpesa element : listone) {
        if(element.getNome().equalsIgnoreCase(
            nomeLista.trim())) {
            return element;
        }
    }
    throw new ElementoNonTrovatoException("lista
    □ inesistente");
}

```

numeroListe: Restituisce il numero totale di liste della spesa.

Restituisce:

- **Il numero di liste.**

```

public static int numeroListe() {
    return listone.size();
}

```

creaCategoria: Crea una nuova categoria.

Parametri:

- **newCategoria:** Il nome della nuova categoria da creare.

Eccezioni:

- **ParametroException:** Se il nome della categoria è "null" oppure vuoto.
- **RidondanzaException:** Se la categoria da inserire è già presente.

```

public static void creaCategoria(String
    newCategoria) throws ParametroException,
    RidondanzaException {
    checkString(newCategoria);
    if(elencoCategorie.contains(newCategoria.
        trim().toLowerCase())) {
        throw new RidondanzaException("categoria
            □ridondante");
    }
    elencoCategorie.add(newCategoria.trim().
        toLowerCase());
}

```

eliminaCategoria: Elimina una categoria e assegna agli articoli appartenenti ad essa la categoria di default.

Parametri:

- **categoria:** Il nome della categoria da eliminare.

Eccezioni:

- **ParametroException:** Se la stringa fornita è "null" oppure vuota.

Restituisce:

- **true:** Se la categoria specificata esisteva.
- **false:** Altrimenti.

```

public static boolean eliminaCategoria(String
    categoria) throws ParametroException {
    checkString(categoria);
    if (!elencoCategorie.removeIf(pippo -> pippo
        .equalsIgnoreCase(categoria))) {
        return false;
    }
    for(ListaSpesa lista : listone) {
        lista.eliminaCategoria(categoria);
    }
}

```

```

    }
    return true;
}

```

containsCategoria: Verifica se la categoria esiste.

Parametri:

- **categoria:** Il nome della categoria da cercare.

Eccezioni:

- **ParametroException:** Se la stringa fornita è "null" oppure vuota.

Restituisce:

- **true:** Se il Gestore contiene la categoria.
- **false:** Altrimenti.

```

public static boolean containsCategoria(String
    categoria) throws ParametroException {
    checkString(categoria);
    return elencoCategorie.contains(categoria.
        toLowerCase().trim());
}

```

numeroCategorie: Restituisce il numero totale di categorie.

Restituisce:

- **Il numero di categorie presenti.**

```

public static int numeroCategorie() {
    return elencoCategorie.size();
}

```

aggiungiArticolo: Aggiunge l'articolo specificato alla lista della spesa indicata.

Parametri:

- **articolo:** L'articolo da aggiungere.
- **nomeLista:** La lista in cui aggiungere l'articolo.

Eccezioni:

- **ParametroException:** Se la stringa fornita è "null" oppure vuota.
- **ElementoNonTrovatoException:** Se la lista indicata non esiste.
- **RidondanzaException:** Se l'articolo è già presente in lista.

```
public static void aggiungiArticolo(  
    ArticoloSpesa articolo, String nomeLista)  
    throws ParametroException,  
    RidondanzaException,  
    ElementoNonTrovatoException {  
    if(!containsCategoria(articolo.getCategoria  
        ()) &&  
        !articolo.getCategoria().equals(  
            ArticoloSpesa.CATEGORIA_DEFAULT)) {  
        throw new ParametroException("categoria_□  
            dell'articolo_□ inesistente");  
    }  
    getListaSpesa(nomeLista).aggiungi(articolo);  
}
```

eliminaArticolo: Elimina l'articolo indicato dalla lista della spesa.

Parametri:

- **articolo:** L'articolo da eliminare.
- **nomeLista:** La lista in cui eliminare l'articolo.

Eccezioni:

- **ParametroException:** Se la stringa fornita è "null" oppure vuota.
- **ElementoNonTrovatoException:** Se la lista specificata non esiste.

Restituisce:

- **true:** Se l'articolo da eliminare era presente in lista.
- **false:** Altrimenti.

```
public static boolean eliminaArticolo(
    ArticoloSpesa articolo, String nomeLista)
    throws ElementoNonTrovatoException,
        ParametroException {
    return getListaSpesa(nomeLista).
        eliminaPerNome(articolo.getNome());
}
```

eliminaArticolo (per nome): Elimina l'articolo per nome dalla lista specificata.

Parametri:

- **articolo:** Il nome dell'articolo da eliminare.
- **nomeLista:** La lista in cui eliminare l'articolo.

Eccezioni:

- **ParametroException:** Se la stringa fornita è "null" oppure vuota.
- **ElementoNonTrovatoException:** Se la lista specificata non esiste.

Restituisce:

- **true:** Se l'articolo da eliminare era presente in lista.
- **false:** Altrimenti.

```

public static boolean eliminaArticolo(String
    nomeArticolo, String nomeLista) throws
    ParametroException,
    ElementoNonTrovatoException {
    return getListaSpesa(nomeLista).
        eliminaPerNome(nomeArticolo);
}

```

containsInLista: Verifica se l'articolo è presente nella lista della spesa specificata.

Parametri:

- **articolo:** L'articolo da cercare.
- **nomeLista:** La lista in cui cercare l'articolo.

Eccezioni:

- **ParametroException:** Se la stringa fornita è "null" oppure vuota.
- **ElementoNonTrovatoException:** Se la lista specificata non esiste.

Restituisce:

- **true:** Se l'articolo è presente in lista.
- **false:** Altrimenti.

```

public static boolean containsInLista(
    ArticoloSpesa articolo, String nomeLista)
    throws ParametroException,
    ElementoNonTrovatoException {
    return getListaSpesa(nomeLista).contains(
        articolo);
}

```

containsInLista (per nome): Verifica se l'articolo con il nome indicato è presente nella lista della spesa.

Parametri:

- **nomeArticolo:** Il nome dell'articolo da cercare.
- **nomeLista:** La lista in cui cercare l'articolo.

Eccezioni:

- **ParametroException:** Se almeno una delle due stringhe fornite è "null" oppure vuota.
- **ElementoNonTrovatoException:** Se la lista specificata non esiste.

Restituisce:

- **true:** Se l'articolo è presente in lista.
- **false:** Altrimenti.

```
public static boolean containsInLista(String
    nomeArticolo, String nomeLista) throws
    ParametroException,
    ElementoNonTrovatoException {
    return getListaSpesa(nomeLista).contains(
        nomeArticolo);
}
```

cercaArticoli: Restituisce tutti gli articoli con il prefisso indicato nella lista specificata.

Parametri:

- **nomeArticolo:** Il prefisso con cui cercare.
- **nomeLista:** La lista in cui cercare gli articoli.

Eccezioni:

- **ParametroException:** Se nomeLista o nomeArticolo sono "null" oppure se nomeLista è vuota.

- **ElementoNonTrovatoException:** Se la lista specificata non esiste.

Restituisce:

- **Tutti gli articoli che hanno nome che inizia per nomeArticolo.**

```
public static ArrayList<ArticoloSpesa>
cercaArticoli(String nomearticolo, String
nomeLista) throws ParametroException,
ElementoNonTrovatoException {
    return getListaSpesa(nomeLista).ricercaNome(
        nomearticolo);
}
```

mytoString: Restituisce una rappresentazione a stringa dell'oggetto.

Restituisce:

- **Una stringa rappresentante l'oggetto.**

```
public static String mytoString() {
    StringBuilder risultato = new StringBuilder
    ();
    risultato.append("Gestore:\nCategorie:_" +
        elencoCategorie.toString() + "\n");

    for(ListaSpesa element : listone) {
        risultato.append(element.toString()).
            append("\n");
    }

    return risultato.toString();
}
```

getListe: Restituisce una copia di tutte le liste della spesa.

Restituisce:

- Una **ArrayList** contenente tutte le **listeSpesa**.

```
@SuppressWarnings("unchecked")
public static ArrayList<ListaSpesa> getListe() {
    return (ArrayList<ListaSpesa>) listone.clone
        ();
}
```

getCategorie: Restituisce una copia di tutte le categorie.

Restituisce:

- Una **ArrayList** contenente tutte le **Categorie**.

```
@SuppressWarnings("unchecked")
public static ArrayList<String> getCategorie() {
    return (ArrayList<String>) elencoCategorie.
        clone();
}
```

serializza: Salva su file tutte le liste della spesa e le categorie. Per poter salvare sia l'elenco di tutte le liste della spesa con il loro contenuto e le categorie create dall'utente ho dovuto sfruttare un **HashMap**, in modo da poter salvare in un solo oggetto due strutture dati contenenti tipi di dati diversi ("String" e "ListaSpesa").

```
public static void serializza() throws
    FileNotFoundException, IOException {
    ObjectOutputStream fbinarioOut = new
        ObjectOutputStream(new FileOutputStream("
            liste.txt"));
    HashMap<ArrayList<ListaSpesa>, ArrayList<
        String>> mario = new HashMap<>();
    mario.put(listone, elencoCategorie);
    fbinarioOut.writeObject(mario);
}
```

```

        fbinarioOut.flush();
        fbinarioOut.close();
    }

```

deserializza: Carica da file le liste della spesa e le categorie salvate. Per recuperare i dati dalla HashTable viene creata una variabile di appoggio "entry" che coincide con la prima riga della HashTable.

```

public static void deserializza() throws
    ClassNotFoundException, IOException,
    ParametroException, RidondanzaException {
    ObjectInputStream fin = new
        ObjectInputStream(new FileInputStream("
        liste.txt"));

    @SuppressWarnings("unchecked")
    HashMap<ArrayList<ListaSpesa>, ArrayList<
        String>> map = (HashMap<ArrayList<
        ListaSpesa>, ArrayList<String>>) fin.
        readObject();
    fin.close();

    Map.Entry<ArrayList<ListaSpesa>, ArrayList<
        String>> entry = (Entry<ArrayList<
        ListaSpesa>, ArrayList<String>>)
    map.entrySet().iterator().next();

    listone.clear();
    elencoCategorie.clear();

    listone.addAll(entry.getKey());
    elencoCategorie.addAll(entry.getValue());
}

```

3 Package "exceptions":

Il seguente pacchetto contiene tutte le eccezioni specifiche del programma. Tutte le eccezioni sono dotate di due costruttori: uno che richiede una stringa da restituire come messaggio dell'eccezione ed uno privo di input che mostra il messaggio di default. ecco un esempio:

```
public class ElementoNonTrovatoException extends
    GenericException{
    private static final long serialVersionUID =
        -103886372164196739L;
    public ElementoNonTrovatoException (String msg){
        super( msg );
    }
    public ElementoNonTrovatoException (){
        super("elemento_non_trovato");
    }
}
```

Tutte le eccezioni sono figlie di "GenericException". La maggior parte dei metodi delle classi astratte possiede la clausola "throws GenericException" per poter permettere alle classi figlie di lanciare eccezioni più specifiche a seconda delle loro necessità. Le eccezioni all'interno del pacchetto sono:

- GenericException: Eccezione generica. Serve alle classi astratte per permettere ai loro figli di lanciare eccezioni più specifiche.
- ElementoNonTrovatoException: Eccezione per segnalare l'assenza di una data istanza in una struttura dati.
- OperazioneNonSupportataException: Eccezione per segnalare che un certo metodo non è supportato dall'implementazione corrente.
- ParametroException: Eccezione per segnalare un errore nei parametri forniti, che sia a causa del loro formato o del loro valore.
- RidondanzaException: Eccezione per segnalare che l'aggiunta dell'oggetto porterebbe ad una ridondanza.

4 Package "interfacciaUtente":

Il Package contiene sia il codice dell'interfaccia grafica che quella da linea di comando.

4.1 Interfaccia Grafica

L'interfaccia grafica è formata da 7 classi, di cui 2 JFrame("InterfacciaGraficaDefinitiva" e "FrameLista") e 5 JPanel (tutte le altre tranne InterfacciaRigaComando).

Per rendere più chiare la posizione delle classi nell'interfaccia ho costruito un albero. La gerarchia va dall'alto al basso, con le classi in alto che impostano le classi sottostanti nel proprio frame/panel nella posizione scritta nel ramo che le collega. Quindi "TextAreaListe" si trova nel centro del frame "InterfacciaGraficaDefinitiva" mentre "ElencoListe" si trova al centro del Panel "TextAreaListe".

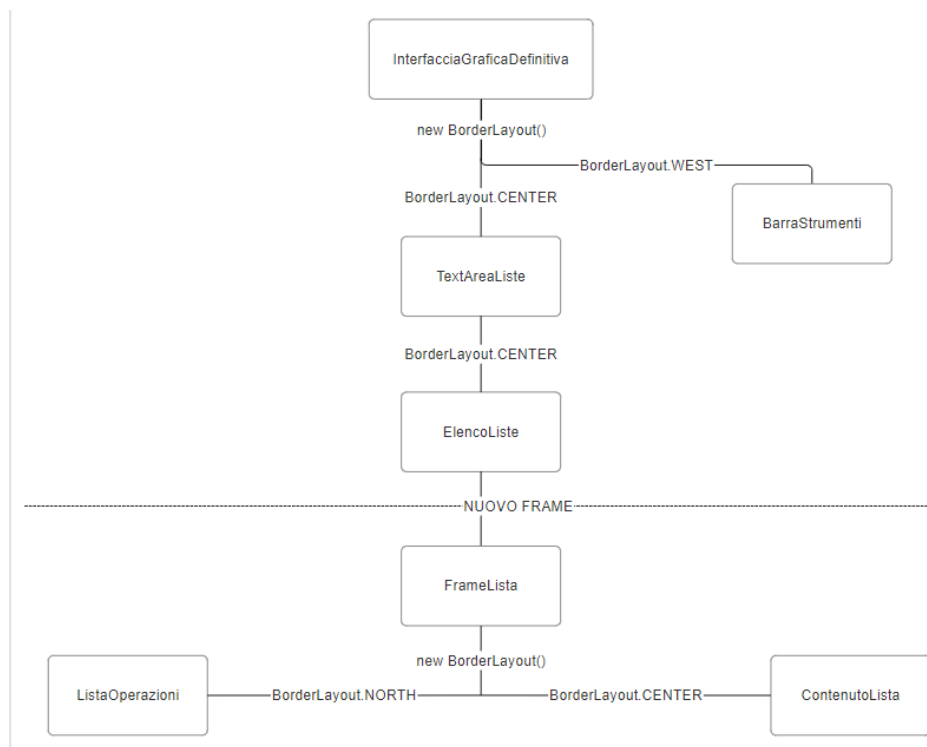


Figure 1: Albero dei frame dell'interfaccia utente

4.1.1 InterfacciaGraficaDefinitiva

E' il Frame visualizzato all'avvio dell'interfaccia. Fornisce gli strumenti di gestione: delle liste della spesa (creazione, eliminazione), delle categorie e di salvataggio e caricamento da file. Sfrutta la disposizione "BorderLayout" al cui interno si trovano:

- WEST: BarraStrumenti.
- CENTER: TextAreaListe.

4.1.2 BarraStrumenti

Contiene 7 bottoni che permettono di operare sulle liste, categorie e sul salvataggio/caricamento da file.

Il risultato delle operazioni viene visualizzato nel JPanel TextAreaListe che si occupa della visualizzazione dei dati.

4.1.3 TextAreaListe

Occupi il centro del Frame. Si occupa della visualizzazione delle liste dell'utente. Sfrutta la disposizione "BorderLayout" al cui interno si trovano:

- PAGE START: il titolo "Liste".
- CENTER: ElencoListe.
- PAGE END: l'elenco delle categorie create dall'utente.

4.1.4 ElencoListe

Sfrutta un layout di tipo "GridLayout". Ad ogni bottone corrisponde una lista. Tutte le funzioni utilizzate da "BarraStrumenti" sulle liste sono prese da questa classe (in modo che le modifiche si ripercuotano visivamente).

Se viene premuto uno dei bottoni, la classe invoca il "FrameLista" che mostra il contenuto della lista relativa e nasconde il frame "InterfacciaUtenteDefinitiva".

4.1.5 FrameLista

Secondo e ultimo frame dell'interfaccia. Si occupa di mostrare e modificare il contenuto di una lista. Sfrutta la disposizione "BorderLayout" al cui interno si trovano:

- NORTH: ListaOperazioni.
- CENTER: ContenutoLista.

4.1.6 ListaOperazioni

Contiene 6 bottoni che permettono di creare, eliminare e filtrare gli elementi della lista. Sfrutta un layout di tipo "FlowLayout".

4.1.7 ContenutoLista

E' una JTable che mostra tutti gli articoli all'interno della lista. Tutte le operazioni eseguite da Panel "ListaOperazioni" sono visualizzate qui. Per modificare i campi degli articoli basta premere due volte sull'articolo da modificare.

4.2 Interfaccia da riga di comando:

Come l'interfaccia grafica, si sviluppa su due "Home" principali, la prima, chiamata appunto "HOME", permette di agire sulle liste, le categorie e il file; la seconda, chiamata "OPERAZIONI LISTA" permette di agire sugli articoli presenti in una lista.

Se in una qualsiasi operazione di inserimento dell'utente, i dati forniti sono inadatti o sbagliati, è richiesto all'utente di inserirli nuovamente.

In ogni momento è possibile premere "Q" per tornare indietro.

5 Main

Il main del progetto. All'avvio del programma l'utente può scegliere se visualizzare l'interfaccia da linea di comando (1) o l'interfaccia grafica (2).